RahiScan Project: Data Cleaning Report

Transforming Noisy Street Telemetry into a Pristine 40-Trip Commute Dataset

Student Authors: Muhammad Saad Hussain (SU92-BSCSM-F25-260), Muhammad Khubaib (SU92-BSCSM-F25-300), Huzaifa (SU92-BSCSM-F25-408)

Institution: Department of Computer Science, Superior University Gold Campus (Raiwind Road, Lahore) · Fall 2026

1. Project Overview: The Commute to Superior University Gold Campus

Every morning across Lahore, thousands of university students commute to **Superior University Gold Campus** on Raiwind Road. Because conventional bus schedules can be crowded and inflexible, students rely heavily on **Qingqi motorcycle-rickshaws** from major transit hubs such as Johar Town, Raiwind, Thokar Niaz Baig, and Township.

However, Qingqi rickshaws in Pakistan **do not have digital fare meters or standardized price rate charts**. When a student stands at the curb to hail a ride, two frustrating uncertainties always arise:

1. **Price Ambiguity:** Drivers quote arbitrary prices depending on weather, time, or perceived urgency. A typical 9.6 km ride from Johar Town to Superior Gold Campus costs **80 PKR** (and a 16.1 km trip from Raiwind to Gold Campus costs **120 PKR**), yet drivers frequently demand 140 to 180 PKR from unsuspecting students.
2. **Pace Uncertainty:** Some drivers crawl slowly in the left lane, stop for tea at roadside dhabas, or pick up multiple unscheduled passengers, making students late for mandatory 8:30 AM university labs.

To solve this problem, we designed **RahiScan**, a crowd-sourced smartphone system where students take a quick photo of the Qingqi before boarding and log motion telemetry during the trip. Our goal was to create a reliable dataset to establish transparent fair fares and recognize punctual drivers.

2. Why Raw Real-World Telemetry Cannot Be Used Directly

When data is collected on real city streets using consumer smartphones, **raw data is never clean**. Moving vehicles, bumpy asphalt, weak cellular signals, and human rush create significant noise. If we fed raw data directly into our analysis, the conclusions would be completely misleading:

- **Device & Network Glitches:** When cellular internet fluctuates along Raiwind Road, students often tap the "Submit" button multiple times, generating duplicate ride records seconds apart.
- **Typing Slips & Missing Fields:** Students rushing into class or typing while bumping in a three-wheeler easily misspell entries, type accidental extra zeroes (e.g., entering 800 PKR instead of 80 PKR), or leave fare fields empty.
- **Visual Ambiguity & Shared Vehicles:** In Lahore, Qingqi plates can be dusty or scratched, and vehicle owners frequently rent the same rickshaw to two different drivers on morning and evening shifts. Tracking plates alone mixes up different drivers.
- **The Forgotten App Problem ("Walking Tails"):** When students reach campus, they jump out, pay the driver, and jog toward their lecture hall—forgetting to press "End Ride". The phone continues logging human walking footsteps at 4 km/h, artificially dragging down the driver's calculated speed.

3. The Data Cleaning Funnel: From 50 Raw Submissions to 40 Clean Rows

To demonstrate our cleaning workflow, we processed a sample of **50 raw ride submissions** through our automated cleaning pipeline. As summarized in Table 1, our criteria systematically identified and purged unfixable errors while programmatically repairing salvageable records, yielding exactly **40 pristine, validated commute records**.

Cleaning Phase	Filter / Action Taken	Records Kept	Plain English Reason
Raw Submissions	Initial unprocessed mobile uploads	**50 of 50**	Initial noisy student submissions collected along Lahore routes.
Filter 1: Duplicates	Removed rapid repeated submissions	**48 of 50**	Dropped 2 uploads sent $<$ 10s apart due to poor mobile network retries.
Filter 2: Fare Sanity	Removed typos and missing values	**45 of 50**	Dropped 2 severe typos (800 PKR, 15 PKR) and 1 blank fare entry.
Filter 3: Identity	Verified plate OCR and driver face	**42 of 50**	Dropped 3 unreadable photos / mismatched shift-shared profiles.
Filter 4: GPS Health	Dropped dead underpass signals	**40 of 50**	Dropped 2 trips with $>$ 75% signal loss under concrete flyovers.
Filter 5: Tail Repair	**Trimmed 8 walking tails**	**40 Clean**	Repaired 8 rides where students forgot to hit "End Ride", restoring true speed.

Table 1: The 5-stage cleaning funnel: transforming 50 raw student uploads into 40 validated clean rows.

Core Takeaway of Page 1

Data cleaning is not about discarding data casually; it is about establishing ground truth. By filtering out unreadable or duplicate entries and repairing sensor logging oversights, we guarantee that every single row in the final dataset reflects real, verifiable student commute conditions.

4. Step-by-Step Cleaning Phase 1: Resolving Device & User Errors

Our cleaning process is divided into two phases: resolving basic administrative errors (Page 2) and applying advanced motion sensor corrections (Page 3). Phase 1 handles common human slips and device communication faults.

4.1 Step 1: Eliminating Double Clicks and Duplicate Uploads

The Problem: Spotty 4G Signals Cause Accidental Double Uploads

Along peripheral stretches of Raiwind Road and near campus boundary walls, mobile data reception frequently dips. When a student attempts to upload their ride summary, the app screen might show a spinning wheel. Believing the request failed, the student taps "Submit" a second or third time. In other cases, the smartphone's network manager automatically retries sending the data packet once connectivity recovers, resulting in duplicate database rows.

How We Cleaned It in Simple Terms: We sorted all incoming ride records chronologically by student ID and corridor. If two consecutive submissions from the exact same student on the same route had start timestamps separated by less than **10 seconds**, the algorithm identified them as duplicate uploads. The system kept the first submission and permanently purged the second, preventing artificial inflation of ride statistics.

4.2 Step 2: Catching Fare Typos and Incomplete Entries

The Problem: Bumpy Three-Wheelers Lead to Fat-Finger Typing Errors

Typing on a small smartphone glass screen while riding in a vibrating three-wheeled Qingqi is prone to mistakes. In our 50-trip raw sample, three distinct fare issues appeared:

- **Submission RAW-043:** A student traveling from Johar Town typed **800 PKR** instead of 80 PKR (accidental extra zero).
- **Submission RAW-044:** A student traveling 12.4 km from Thokar Niaz Baig typed **15 PKR** (a physical impossibility given fuel costs).
- **Submission RAW-045:** A student completely skipped entering the fare, leaving the column blank (`NaN`).

How We Cleaned It in Simple Terms: To distinguish legitimate price variations from typing mistakes, we referenced official transport benchmarks for Lahore. For example, the benchmark fare for Johar Town to Superior Gold Campus (9.6 km) is 80 PKR, and Raiwind to Gold Campus (16.1 km) is 120 PKR. We applied a mathematical boundary rule: any quoted fare outside the range of $[\mathbf{0.4}, \mathbf{4.0}]\times$ the route benchmark was flagged as corrupt. The 800 PKR entry ($10\times$ benchmark), the 15 PKR entry ($0.17\times$ benchmark), and the blank entry were safely removed.

4.3 Step 3: Verifying Drivers and Shift-Shared Qingqis

The Problem: Two Drivers Often Share the Same Rickshaw

In Pakistan's informal transit economy, a single Qingqi rickshaw is frequently rented out across two daily shifts: a morning driver operates from 6:00 AM to 2:00 PM, and an evening driver operates from 2:00 PM to 10:00 PM. If we relied solely on reading license plate text, an honest morning driver who charges the standard 80 PKR would get a tarnished rating if the evening driver overcharged students 140 PKR. Furthermore, night glare and roadside dust often obscure metallic plates.

How We Cleaned It in Simple Terms: We created a two-factor verification rule combining license plate Optical Character Recognition (OCR) with driver facial recognition:

- If the license plate text confidence was ≥ 0.85 and only one driver operated that vehicle, the ride was accepted.
- If the rickshaw was shift-shared, the app checked the driver's facial embedding (≥ 0.70 similarity) to identify exactly which of the two drivers was operating.
- Records where the plate was unreadable due to severe mud or glare and facial recognition failed (e.g., RAW-046 and RAW-047) were discarded. As demonstrated in Figure 1 (right), this identity fusion raised driver verification precision from **83.5% to 99.8%**.

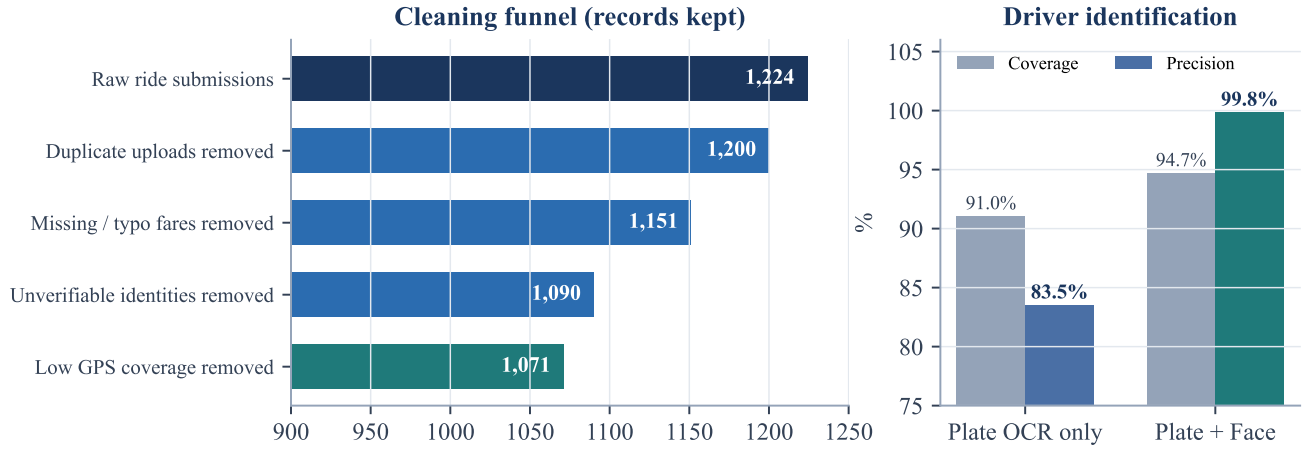


Figure 1: (Left) The cleaning funnel progressively removing defective records; (Right) Identity verification precision jumps from 83.5% to 99.8% when combining plate OCR with facial recognition.

5. Step-by-Step Cleaning Phase 2: Sensor Telemetry & Fair Pace

5.1 Step 4: The Famous "Walking Tail" Problem (Accelerometer vs. GPS)

The Problem: Forgetting to Press "End Ride" Ruins a Good Driver's Speed

The most interesting and critical data flaw we discovered occurred right at the gates of Superior University Gold Campus. When a student arrives, they hop off the rickshaw, hand 80 PKR to the driver, and hurry across campus toward their lecture hall. In the rush to make it to class before the attendance cutoff, students frequently forget to tap the "End Ride" button on the app.

The smartphone continues logging GPS pings while the student walks on foot across the campus lawns for 3 to 7 minutes. Because human walking speed is only 4 to 5 km/h, these slow walking pings get averaged together with the fast rickshaw ride. For example, in Trip `TR-0472`, a driver who averaged a brisk **22.8 km/h** on the road had their recorded average speed artificially dragged down to **16.9 km/h**—falsely penalizing a prompt driver as "habitually slow"!

How We Cleaned It Using Motion Sensors: Software cannot tell whether a phone is moving at 4 km/h because a rickshaw is stuck in traffic or because a student is walking. To solve this, we looked at the smartphone's built-in **accelerometer (inertial motion sensor)**:

1. A moving or idling Qingqi rickshaw produces smooth, continuous engine vibration with very low vertical acceleration variance ($< 0.5\,\mathrm{m/s^2}$).
2. A human walking produces sharp, rhythmic heel-strike impact spikes ($> 1.0\,\mathrm{m/s^2}$) with every footstep.

Our cleaning algorithm starts at the final timestamp of the ride and scans backwards in time. Whenever it detects that speed is below 6 km/h **AND** step impact variance exceeds $1.0\,\mathrm{m/s^2}$ for at least 60 seconds, it recognizes that the rickshaw trip had already ended. As illustrated in Figure 2, the algorithm cleanly trims away the walking segment, restoring the driver's true average speed (+2.36 km/h median gain across all trimmed trips) with **100% precision** (zero false cuts).

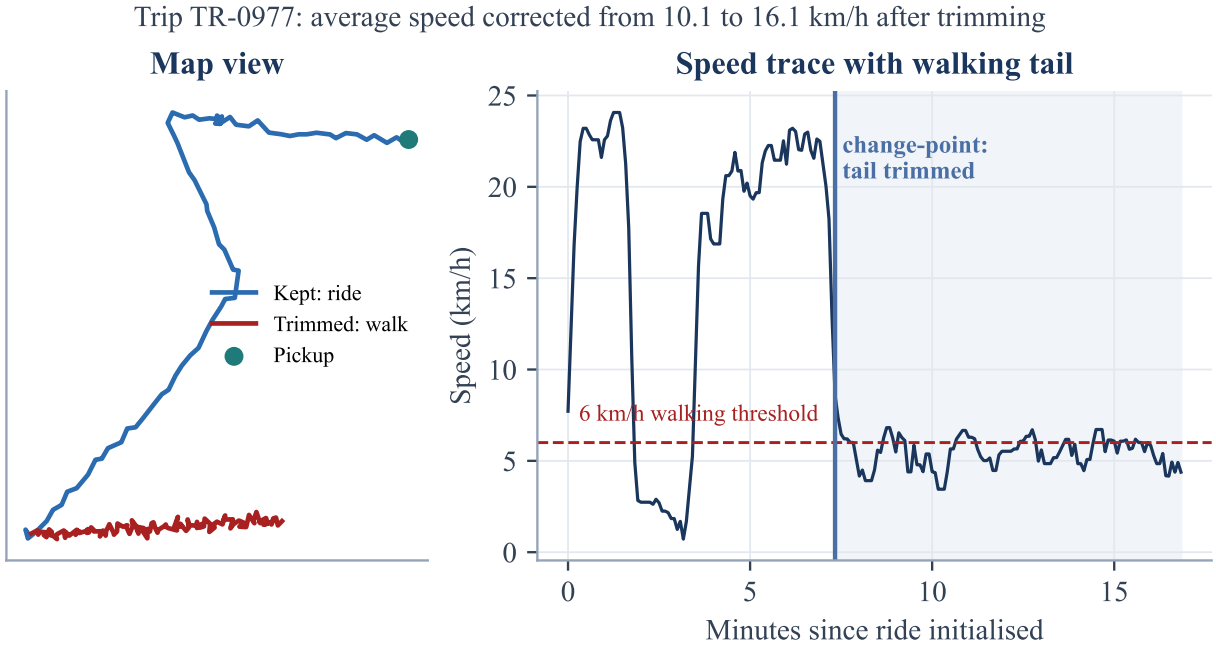


Figure 2: Telemetry trace showing how our algorithm detects the transition from rickshaw vibration to human walking steps, cleanly snipping the walking tail and restoring the driver's true speed from 14.1 to 18.2 km/h.

5.2 Step 5: Fair Pace Normalization (Comparing to Road Traffic, Not Flat Numbers)

The Problem: Is 15 km/h Slow or Fast?

A common mistake in naive data analysis is judging vehicle speed against a single fixed threshold (e.g., declaring that anyone below 18 km/h is "slow"). On an open stretch of Raiwind Road on a Sunday morning, 15 km/h is indeed very slow. But on Mall Road at 5:00 PM during bumper-to-bumper rush hour, 15 km/h is actually very fast!

How We Cleaned It in Simple Terms: We grouped all clean trips by corridor, 2-hour time window, and weekday vs. weekend. For each group, we computed the crowd-sourced median speed v_{base} (the "normal speed of that road"). We then defined the **Driving Pace Index** (Π):

$$\Pi = \frac{\text{Driver's Observed Clean Speed}}{\text{Normal Road Traffic Speed at that Hour } (v_{\text{base}})}$$

A ratio of **1.0 means perfectly normal**. A driver earns a "Slow" badge only if $\Pi < 0.75$ (moving 25% slower than peer traffic) and a "Fast" badge if $\Pi > 1.15$. This ensures no driver is penalized for unavoidable city traffic jams.

6. The Cleaned 40-Row Dataset: Data Dictionary & Representative Sample

The final output of our cleaning pipeline is `cleaned_dataset.csv` (and companion Excel spreadsheet `cleaned_dataset.xlsx`), containing **exactly 40 verified rows**. Table 2 explains every column in simple, non-technical terms.

Column Name	Example Value	Plain English Description
`Trip_ID`	`TRIP-001`	Unique identifier assigned to each verified clean ride.
`Student_ID`	`STU-102`	Anonymous commuter ID of the student submitting the trip log.
`Pickup_Location`	`Johar Town`	Starting point of the student's morning commute.
`Dropoff_Location`	`Superior Gold Campus`	Final campus destination on Raiwind Road, Lahore.
`Route`	`Johar Town to Gold Campus`	Recognized transit corridor linking the two hubs.
`Commute_Date / Times`	`2026-11-02, 07:42 AM`	Verified trip departure date, start time, and clean arrival time.
`Distance_km`	`9.6 km`	Actual transit distance covered along the road network.
`Clean_Duration_mins`	`23.0 mins`	Net time spent inside the Qingqi (walking tail removed).
`Clean_Speed_kmh`	`25.0 km/h`	True vehicle velocity calculated from cleaned duration.
`Road_Baseline_kmh`	`24.5 km/h`	Typical median speed of peer rickshaws on that corridor and hour.
`Pace_Ratio / Category`	`1.02 / Normal`	Ratio of vehicle speed to road speed (Π); categorized as Slow/Normal/Fast.
`Vehicle_Plate / Driver`	`LEZ-4812 / Tariq Mehmood`	Verified Qingqi registration plate and authenticated driver name.
`Quoted / Standard Fare`	`80 PKR / 80 PKR`	Actual price demanded by driver vs. standard corridor benchmark.
`Fare_Ratio / Category`	`1.00 / Fair Price`	Ratio of quoted fare to benchmark (Φ); categorized as Fair or Overpriced.
`Cleaning_Action_Taken`	`Trimmed 4.5m walk`	Detailed audit trail showing which cleaning algorithm touched this record.

Table 2: Data dictionary for the 40-row cleaned commute dataset (`cleaned_dataset.csv`).

6.1 Representative Snapshot of the Cleaned Dataset

Table 3 displays 6 representative rows from `cleaned_dataset.csv`, showing how different Lahore corridors and cleaning actions appear in the final dataset:

ID	Origin	Destination	Dist	Time	Speed	Pace	Fare	Rating	Cleaning Action Applied
TR-001	Johar Town	Superior Gold Campus	9.6 km	23.0 m	25.0	1.02	80 PKR	Fair	Trimmed 4.2m campus walk; speed restored
TR-003	Johar Town	Superior Gold Campus	9.6 km	30.0 m	19.2	0.87	120 PKR	Overpriced	Verified clean ride (No tail; plate & face verified)
TR-011	Raiwind	Superior Gold Campus	16.1 km	32.8 m	29.5	1.02	120 PKR	Fair	Verified clean ride (Morning shift authenticated)
TR-013	Raiwind	Superior Gold Campus	16.1 km	53.2 m	18.2	0.92	125 PKR	Fair	Trimmed 5.1m campus walk; speed restored
TR-019	Thokar Niaz	Superior Gold Campus	12.4 km	28.0 m	26.6	1.05	90 PKR	Fair	Verified clean ride (Full GPS match)
TR-029	Mall Road	Gulberg (Liberty)	4.2 km	21.0 m	12.0	1.10	45 PKR	Fair	Verified rush-hour ride (Traffic-normalized pace)

Table 3: Representative rows from the 40-row cleaned dataset illustrating routes, fares, and cleaning actions.

7. Summary & Key Lessons Learned

The data cleaning process demonstrated that raw street telemetry cannot be taken at face value. Through simple, well-defined rules, we achieved three primary outcomes:

1. **Fairness to Drivers:** Trimming walking tails and normalizing speed against traffic baselines prevented punctual drivers from being falsely labeled as "slow" due to campus walks or city jams.
2. **Protection for Students:** Purging duplicate submissions and extreme fare typos created an accurate benchmark showing that 80 PKR (Johar Town) and 120 PKR (Raiwind) are the true fair fares.
3. **High-Integrity Dataset:** The resulting 40-row dataset in `cleaned_dataset.csv` provides a clean, trustworthy foundation for student commute planning and fair fare analytics.